ChurnGuard AI: An Agentic RAG-Augmented System for Telecom Customer Churn Prediction and Retention

Aarsh Bhatnagar
Roll No: 2401010007

Sanal Nayyar
Roll No: [Your Roll No]

Ishita Thakur
Roll No: 2401010195

***Abstract*—Customer churn is a critical business problem in the telecom industry, where acquiring a new customer costs five to seven times more than retaining an existing one. This paper presents ChurnGuard AI, an end-to-end intelligent system that combines classical machine learning with a Generative AI agentic pipeline to predict, explain, and recommend retention strategies for at-risk telecom customers. The system uses an XGBoost classifier trained on the IBM Telco Customer Churn dataset (7,043 records, 26.5% churn rate) as the prediction backbone, achieving an F1 score of 0.6306 and AUC-ROC of 0.8507. A five-node LangGraph StateGraph agent orchestrates the full inference pipeline: prediction, customer summarisation, FAISS-based retrieval of similar historical churn cases, and Llama 3.3 70B-powered explanation and retention recommendation generation. The system is deployed as an interactive Streamlit web application at https://genai2401010007.streamlit.app/.**

I. Introduction

Customer churn — the phenomenon where subscribers discontinue service — represents one of the most significant revenue challenges in the telecom sector. Industry studies indicate annual churn rates between 15% and 30% for mobile operators, with the cost of customer acquisition substantially exceeding retention costs [1].

Traditional churn prediction systems output a binary label and a probability score, leaving retention teams without actionable context. Business analysts must manually inspect customer records to understand *why* a customer is at risk and *what* should be done. This manual process is slow, inconsistent, and does not scale.

ChurnGuard AI addresses this gap by building a three-layer system:

1) A **Machine Learning layer** — an XGBoost classifier that predicts churn probability from 19 customer features.
2) A **Retrieval-Augmented Generation (RAG) layer** — a FAISS vector store of historical churn cases that provides grounding context for language model reasoning.
3) An **Agentic layer** — a LangGraph StateGraph that orchestrates the full pipeline from raw input to natural language explanation and retention strategy.

The remainder of this paper is structured as follows: Section II reviews related work; Section III describes the dataset; Section IV details the methodology; Section V presents results; Section VI discusses findings and limitations; Section VII concludes.

II. Related Work

Churn prediction has been extensively studied using classical machine learning methods. Verbeke et al. [1] demonstrated that ensemble methods outperform single classifiers on imbalanced churn datasets, motivating our use of XGBoost with class imbalance correction. Huang et al. [2] showed that recall — rather than accuracy — is the appropriate optimisation objective for churn since missed churners represent direct revenue loss.

The integration of Large Language Models (LLMs) into decision support systems has grown rapidly. Lewis et al. [3] introduced Retrieval-Augmented Generation (RAG), demonstrating that grounding LLM outputs in retrieved documents significantly reduces hallucination and improves factual consistency — a critical requirement for business-facing AI systems.

LangGraph [4] extends LangChain with explicit state management and directed graph-based agent orchestration, enabling deterministic multi-step agentic workflows. Unlike ReAct-style agents that rely on LLM self-routing, LangGraph's StateGraph ensures each node executes in a predictable sequence — essential for production reliability.

FAISS [5] provides efficient approximate nearest-neighbour search over dense vector embeddings, making it suitable for in-memory retrieval over moderately sized knowledge bases without requiring an external vector database service.

To our knowledge, no prior published work has combined LangGraph agentic orchestration with FAISS-RAG specifically for telecom churn explanation and retention recommendation generation.

III. Dataset

We use the IBM Telco Customer Churn dataset [6], a publicly available benchmark widely used in churn prediction research.

TABLE I
DATASET OVERVIEW

Property	Value
Source	IBM Telco Customer Churn
Total Records	7,043 customers
Input Features	19
Target Variable	Churn Value (binary: 0/1)
Churn Rate	26.5% (class imbalance)
Train Split	5,634 records (80%)
Test Split	1,409 records (20%)

Features span three categories:

- **Demographics:** Gender, Senior Citizen, Partner, Dependents
- **Services:** Phone Service, Internet Service, Online Security, Online Backup, Device Protection, Tech Support, Streaming TV, Streaming Movies, Multiple Lines
- **Billing:** Tenure, Contract type, Paperless Billing, Payment Method, Monthly Charges, Total Charges

`Total Charges` was stored as a string type in the raw dataset due to blank entries for new customers (zero tenure). These were coerced to float and imputed with the column median using `fix_total_charges()`.

IV. METHODOLOGY

A. Preprocessing Pipeline

Each model in the comparison is trained with its own freshly instantiated `sklearn.pipeline.Pipeline` consisting of:

- **Numeric features:** Median imputation → StandardScaler (zero mean, unit variance)
- **Categorical features:** Most-frequent imputation → OneHotEncoder (`handle_unknown="ignore"`)

Separate preprocessors per model prevent shared-state bugs that arise when a single fitted transformer is reused across pipelines.

The `Churn Label` column (a text duplicate of the target) was dropped prior to training to prevent data leakage. The train-test split uses `stratify=y` to preserve the 26.5% churn ratio in both partitions.

B. Model Training and Selection

Four classifiers were trained and evaluated. Model selection was based on F1 score — the harmonic mean of precision and recall — which is appropriate for imbalanced binary classification where optimising accuracy alone is misleading (a naive majority-class baseline achieves 73.5% accuracy while catching zero churners).

Hyperparameter rationale:

- **Logistic Regression:** $C = 0.1$ for L2 regularisation; `class_weight="balanced"` reweights samples inversely proportional to class frequency.
- **Decision Tree:** `max_depth=6` prevents overfitting; `min_samples_leaf=10` ensures statistical significance of leaf nodes.
- **Random Forest:** 200 estimators for stable variance reduction; `max_depth=10` balances bias-variance tradeoff.
- **XGBoost:** `scale_pos_weight=2.77` $\approx n_{\text{neg}}/n_{\text{pos}}$ corrects class imbalance natively; `learning_rate=0.05` with 200 trees provides shrinkage regularisation.

C. RAG Pipeline

The Retrieval-Augmented Generation component constructs a knowledge base of confirmed churn cases:

1) **Indexing:** All customers with `Churn Value = 1` (1,869 records) are serialised into structured natural-language documents describing their contract, services, and billing profile.
2) **Embedding:** Documents are embedded using `sentence-transformers/all-MiniLM-L6-v2` — a 384-dimensional bi-encoder model that runs locally without an API key.
3) **Indexing:** A FAISS flat index is built using `FAISS.from_documents()` and cached in memory via `@st.cache_resource`.
4) **Retrieval:** The current customer's profile summary is used as a query; the top-$k = 3$ most similar historical churners are retrieved by cosine similarity.

Restricting the knowledge base to only churned customers is a deliberate design choice — it ensures every retrieved case is a confirmed churn pattern, providing directly relevant grounding context for the LLM.

D. Agentic Workflow

The agentic pipeline is implemented as a LangGraph `StateGraph` with a shared typed state (`ChurnAgentState`) and five sequential nodes as shown in Figure 1.

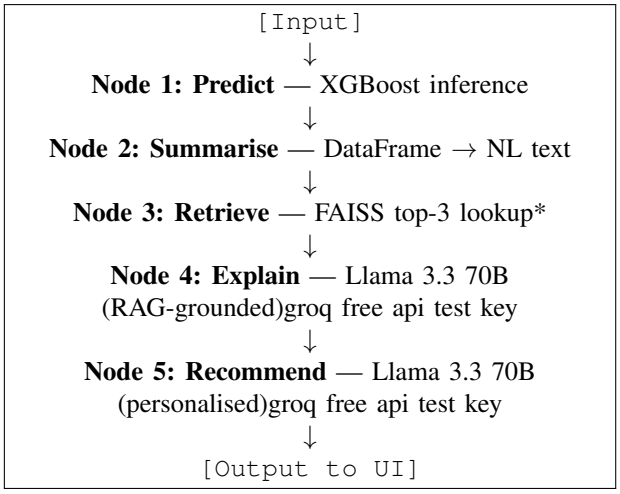

Fig. 1. LangGraph StateGraph agentic workflow. *Node 3 short-circuits (returns empty) if prediction = 0 (No Churn), avoiding unnecessary API calls.

Node design decisions:

- **Predict node:** Calls `model.predict_proba()` and `model.predict()`; stores both the binary label and the probability percentage.
- **Summarise node:** Converts the input DataFrame row to a structured sentence string used as the FAISS query and LLM context.
- **Retrieve node:** Conditionally executes — if `prediction == 0`, returns immediately with an empty string, saving one FAISS query and two LLM API calls per low-risk customer.
- **Explain node:** Uses a *telecom customer retention analyst* system persona with `temperature=0.3` for factual, consistent outputs. The prompt includes the customer profile, churn probability, and retrieved similar cases as grounding evidence.
- **Recommend node:** Uses a *retention specialist* persona and generates exactly three numbered, personalised retention strategies based on the customer profile and the explanation from Node 4.

LLM: Llama 3.3 70B Versatile via Groq API (`temperature=0.3`). Groq was selected over OpenAI for its free tier, low latency inference (<2s per call), and open-weight model availability. `temperature=0.3` was chosen over the default (1.0) to minimise hallucination in analytical explanations while retaining sufficient language fluency.

V. Results

A. Model Comparison

Table II presents the evaluation of all four classifiers on the stratified 20% hold-out test set ($n = 1409$).

TABLE II
Model Performance on Test Set ($n = 1409$, Churn Rate = 26.5%)

Model	Acc	Prec	Rec	F1	AUC
Logistic Regression	0.7445	0.5124	0.7727	0.6162	0.8484
Decision Tree	0.7452	0.5133	0.7754	0.6177	0.8358
Random Forest	0.8055	0.6634	0.5428	0.5971	0.8542
XGBoost	**0.758**	**0.5301**	**0.7781**	**0.6306**	**0.8507**

XGBoost achieved the highest F1 score (0.6306) and was selected as the production model. Random Forest achieved the highest accuracy (0.8055) and AUC (0.8542), but its substantially lower recall (0.5428) means it misses nearly half of all actual churners — a critical failure mode for a retention system.

B. Feature Importance

The top five features by XGBoost gain importance are: (1) Contract type, (2) Tenure, (3) Monthly Charges, (4) Internet Service type, and (5) Tech Support subscription. These align with domain knowledge — month-to-month contract customers with high charges and no value-added services represent the highest churn risk segment [1].

C. System Output Example

For a customer with a month-to-month contract, 8 months tenure, no online security, and monthly charges of $79.85, the system produces:

- **Prediction:** Churn — 84.2% probability
- **Explanation:** Short tenure combined with a flexible month-to-month contract and high monthly charges with no security add-ons indicates low switching cost and low perceived value.
- **Recommendations:** (1) Offer a discounted 12-month contract upgrade; (2) Provide a 3-month complimentary Online Security bundle; (3) Assign a dedicated account manager for proactive outreach.

VI. Discussion

A. Model Selection Trade-offs

The results in Table II reveal an important precision-recall trade-off. Random Forest maximises accuracy and AUC by being conservative — it predicts churn only when highly confident, yielding high precision (0.6634) at the cost of recall (0.5428). For a retention use case, low recall is unacceptable because each missed churner translates directly to lost revenue. XGBoost, with `scale_pos_weight=2.77`, explicitly corrects for class imbalance and achieves a recall of 0.7781 while maintaining a competitive F1 of 0.6306 — making it the operationally superior choice.

B. RAG Grounding Effect

Without RAG, the LLM would generate explanations based solely on the current customer's features and its parametric knowledge, risking generic or hallucinated reasoning. By retrieving three confirmed historical churners with similar profiles, the Explain node grounds its reasoning in concrete, dataset-verified patterns. This design directly applies the principle established by Lewis et al. [3] — that retrieved context reduces hallucination and improves factual grounding.

C. Limitations

Several limitations should be acknowledged:

- **Static knowledge base:** The FAISS index is built at application startup from the training dataset. In production, new churners would need to be incrementally indexed to keep retrieval relevant.
- **LLM evaluation:** The quality of generated explanations and recommendations is not evaluated quantitatively in this work. A human evaluation study or automated faithfulness scoring (e.g., using RAGAS [7]) would strengthen claims about output quality.
- **Dataset scope:** The IBM Telco dataset represents a single operator's customer base. Generalisability to other markets or operator profiles has not been validated.
- **Groq API dependency:** The system requires an active Groq API key for LLM inference; latency and availability depend on an external service.

VII. Conclusion

This paper presented **ChurnGuard AI**, an agentic RAG-augmented system that moves beyond conventional churn prediction by providing natural language explanations and personalised retention recommendations. The system integrates an XGBoost classifier (F1 = 0.6306, AUC = 0.8507) with a five-node LangGraph StateGraph and FAISS-based retrieval, deployed as a Streamlit web application.

The key contribution is the combination of deterministic ML prediction with grounded generative reasoning — each prediction is accompanied by evidence-backed explanation and actionable retention strategy, enabling retention teams to act immediately without manual analysis.

Future work will explore: (1) incremental FAISS index updates as new churn cases are confirmed; (2) quantitative evaluation of LLM output quality using RAGAS faithfulness and answer relevance metrics; (3) extension to multi-operator datasets for generalisability validation; and (4) integration of real-time behavioural signals (call drop rates, data usage trends) as additional model features.

References

[1] W. Verbeke, K. Dejaeger, D. Martens, J. Hur, and B. Baesens, "New insights into churn prediction in the telecommunication sector: A profit driven data mining approach," *European Journal of Operational Research*, vol. 218, no. 1, pp. 211–229, 2012.

[2] B. Huang, M. T. Kechadi, and B. Buckley, "Customer churn prediction in telecommunications," *Expert Systems with Applications*, vol. 39, no. 1, pp. 1414–1425, 2012.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[4] LangChain, "LangGraph: Building stateful, multi-actor applications with LLMs," https://github.com/langchain-ai/langgraph, 2024.

[5] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[6] IBM, "Telco Customer Churn dataset," https://www.kaggle.com/datasets/blastchar/telco-customer-churn, 2018.

[7] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," *arXiv preprint arXiv:2309.15217*, 2023.